

离线网页普通话—粤语文本翻译引擎技术方案

1. 推荐部署形态

建议将应用制作成一个可安装的 **PWA离线网页应用**：

1. 用户首次通过HTTPS打开网页；
2. 浏览器下载网页、翻译引擎和词库；
3. Service Worker将全部必要资源缓存到设备；
4. 用户之后断网仍可打开和使用；
5. 有网络时可选择更新词库和程序。

Service Worker可以拦截网页资源请求，并在离线时从本地缓存返回资源；PWA还可以通过Web App Manifest安装到桌面或手机主屏幕。Service Worker必须运行在HTTPS安全环境中，本地开发时可使用 `localhost`。因此，不建议把正式版本设计成双击 `file:///index.html` 运行。

两种交付模式

模式A：首次联网安装，之后离线

最适合普通用户：

HTTPS网站
↓ 首次访问
安装PWA并缓存资源
↓
永久离线使用

模式B：从未联网的封闭环境

如果设备从一开始就不能联网，可以把同一套网页文件部署到：

- 局域网静态服务器；
- 设备内置的轻量HTTP服务器；
- 企业终端的本地Web服务器；
- WebView或桌面壳程序。

浏览器访问：

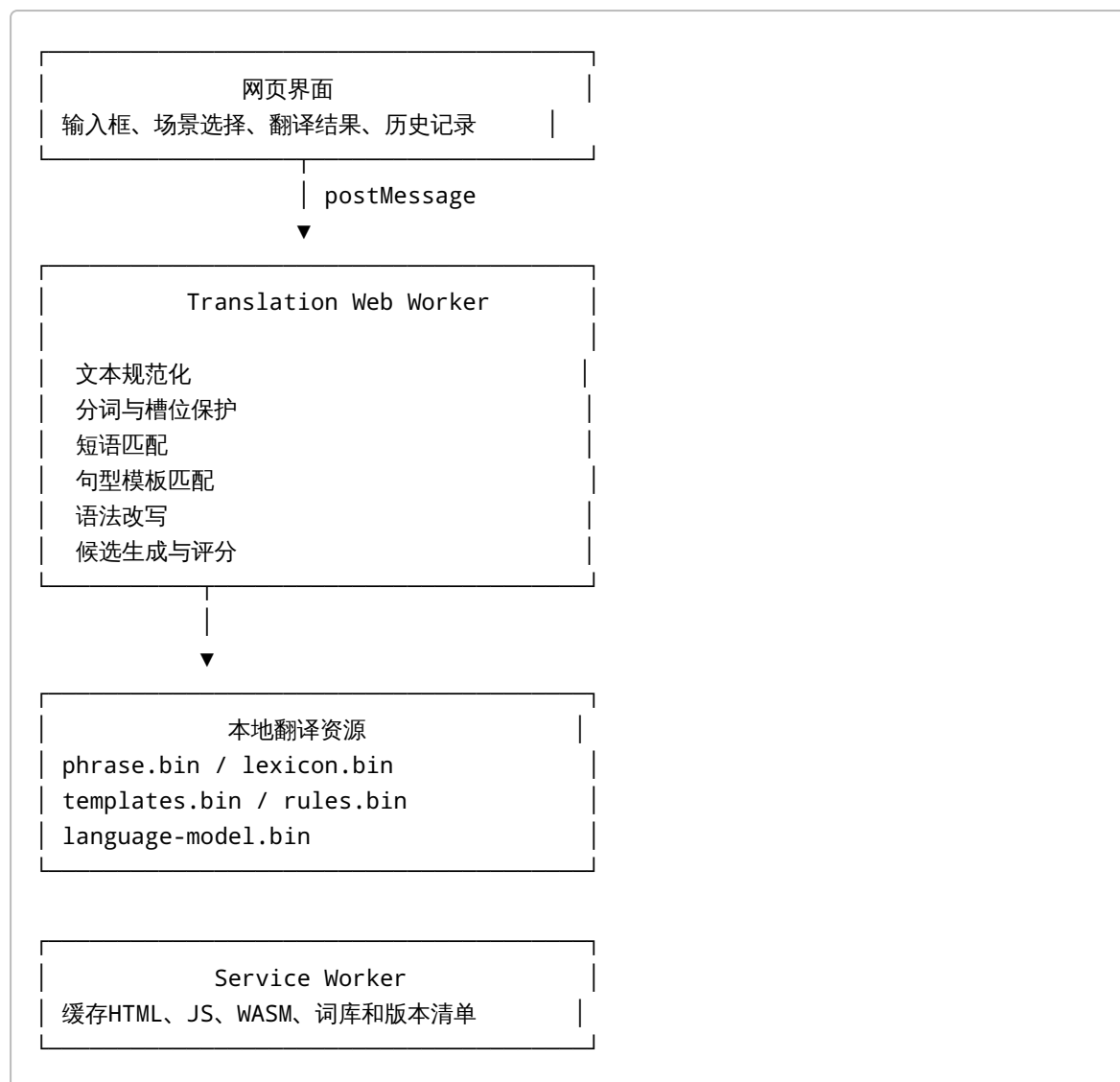
`http://localhost:端口`

而不是直接打开：

`file:///应用目录/index.html`

翻译引擎代码本身保持不变。

2. 整体架构



翻译运算放入Web Worker，避免大词库初始化、候选搜索和规则执行阻塞界面。Web Worker运行在独立后台线程，并通过消息与主页面通信。

3. MVP技术选型

第一版推荐：

模块	实现
前端界面	TypeScript+轻量前端框架或原生DOM
离线能力	Service Worker+Cache Storage

模块	实现
翻译线程	Dedicated Web Worker
翻译引擎	TypeScript
固定词库	压缩二进制文件
运行时索引	Trie+哈希表+有限状态规则
用户数据	IndexedDB
更新机制	版本清单+增量数据包
构建工具	Vite或等价静态构建工具
部署	HTTPS静态网站/localhost服务器

第一版不必立即使用SQLite或WebAssembly。对于约10,000个翻译单元，经过编译的词库完全可以启动时装入内存，直接使用Trie和哈希索引查询。

4. 浏览器内各组件的职责

4.1 Service Worker

Service Worker只负责应用离线和资源更新，不参与翻译逻辑。

缓存内容包括：

```
index.html
app.js
app.css
translator-worker.js
phrase.bin
lexicon.bin
templates.bin
rules.bin
scoring.bin
version.json
```

建议采用两类缓存：

```
app-shell-v12
translation-data-v37
```

这样更新界面时不必重复更新词库，更新词库时也不必重新下载全部网页代码。

Cache Storage适合保存网页资源和不可变数据包，缓存内容的版本切换和删除策略需要由应用自己管理。

4.2 主页面线程

主线程只执行轻量任务：

- 接收用户输入；
- 场景选择；
- 向Worker发送翻译请求；
- 展示翻译结果；
- 显示命中的表达和置信度；
- 管理复制、收藏和历史记录。

主线程不直接加载和遍历完整词库。

4.3 Translation Worker

Worker启动时完成：

1. 加载翻译数据包
2. 解压数据
3. 构建或恢复内存索引
4. 初始化规则执行器
5. 返回 ready 状态

翻译时执行：

```
normalize()  
protectSlots()  
segment()  
matchPhrases()  
matchTemplates()  
applyGrammarRules()  
generateCandidates()  
scoreCandidates()  
restoreSlots()  
postProcess()
```

5. 翻译引擎内部结构

5.1 输入规范化

处理：

- 简繁统一；
- 标点统一；
- 数字和时间统一；
- 连续空格清理；

- 常见口语缩写；
- 人名、地点、金额、网址和英文保护。

例如：

输入：
我明天下午3点去广州南站，你帮我叫辆车

规范化：
我 明天 下午 [TIME:3点] 去 [PLACE:广州南站]，
你 帮 我 叫 一 辆 车

槽位信息不参与普通词汇转换。

5.2 多粒度切分

不要依赖传统通用中文分词结果，而要使用翻译词库驱动的分切。

优先级：

完整句
> 长固定短语
> 句型组成部分
> 普通词语
> 单字

例如：

我 / 今天 / 来不及 / 吃饭

不能切成：

我 / 今天 / 来 / 不及 / 吃饭

实现方式：

- Trie最长匹配；
- 动态规划选择全句最优切分；
- 对“还没有、来不及、看一下、为什么”等高风险结构提高权重。

5.3 短语匹配器

使用Trie保存普通话源短语：

为什么
为什么不
还没有
还没有吃饭
来不及
能不能
麻烦你
帮我看一下

每个短语节点保存：

```
interface PhraseEntry {  
    id: number;  
    source: string;  
    target: string;  
    scene: number;  
    priority: number;  
    leftCondition?: number;  
    rightCondition?: number;  
    flags: number;  
}
```

短语匹配输出的不是立即替换后的字符串，而是中间表示：

[PRONOUN:我]
[TIME:今天]
[PHRASE:来不及 → 赶唔切]
[VERB:吃 → 食]
[NOUN:饭 → 饭]

这样可以避免前面的替换破坏后续规则。

5.4 句型模板引擎

模板编译为有限状态结构，而不是运行时使用大量正则表达式。

源模板：

你为什么{VP}

目标模板：

你点解{VP}

编译后：

```
{
  "tokens": [
    {"literal": "你"},
    {"literal": "为什么"},
    {"slot": "VP"}
  ],
  "output": [
    {"literal": "你"},
    {"literal": "点解"},
    {"slot": "VP"}
  ]
}
```

支持的第一版槽位只保留：

NP	名词短语
VP	动词短语
PERSON	人物
PLACE	地点
TIME	时间
NUMBER	数量
ITEM	商品或物品
ADJ	形容词

模板不要允许任意递归嵌套，以保持离线执行速度和规则可解释性。

5.5 语法改写器

语法规则操作中间表示，不直接反复修改字符串。

例如：

普通话结构：
[还没有] [完成任务]

转换：
[仲未] [完成任务]

普通话结构：
[正在] [开会]

转换：
[开] [紧] [会]

普通话结构：
[比] [昨天] [贵]

转换：
[贵] [过] [寻日]

规则结构：

```
interface RewriteRule {  
    id: number;  
    stage: number;  
    priority: number;  
    pattern: PatternToken[];  
    replacement: OutputToken[];  
    conditions?: RuleCondition[];  
}
```

按照固定阶段执行：

10：否定
20：疑问
30：完成与进行体
40：比较和程度
50：代词及指示词
60：词汇转换
70：语序调整
80：量词和搭配
90：句末语气

同一阶段中按优先级从高到低执行。

5.6 候选生成

不要只生成一个中间结果。对于有歧义的结构，保留少量候选。

例如：

我下班以后去找你

候选：

我放工之后去搵你
我收工之后去搵你
我放工以后去搵你

建议Beam Search宽度控制在：

4–8个候选

日常短句不需要很大的搜索空间。

5.7 候选评分

评分可以全部在浏览器内完成：

```
score =  
  phraseCoverage  
+ templateCoverage  
+ lexicalNaturalness  
+ collocationScore  
+ sceneScore  
+ sentenceEndingScore  
- untranslatedRisk  
- ruleConflict  
- unnaturalSequence
```

第一版使用人工权重：

```
const weights = {  
  exactSentence: 100,  
  phraseCoverage: 30,  
  templateMatch: 25,  
  commonCollocation: 12,  
  sceneMatch: 8,  
  standardCantoneseWord: 6,  
  riskyMandarinResidue: -25,  
  ruleConflict: -30,  
  duplicateParticle: -20  
};
```

后续可以增加一个小型N-gram语言模型，用于判断：

放工之后

是否比：

放工以后

更符合目标粤语口语语料。

该模型可以保存为量化后的二进制频率表，不需要神经网络。

6. 翻译词库的浏览器格式

不建议直接把10,000条数据放入一个大型JSON文件。JSON字段名重复多，解析时还会产生较多临时对象。

推荐构建阶段将编辑数据编译为以下文件：

```
phrase.bin
lexicon.bin
template.bin
rules.bin
ngram.bin
strings.bin
metadata.json
```

其中：

- `strings.bin` 保存所有去重字符串；
- 其他文件通过整数ID引用字符串；
- 数值字段使用定长或变长整数；
- 文件可使用Brotli或gzip压缩；
- 浏览器加载后转换为 `ArrayBuffer` 和 `TypedArray`；
- Trie节点尽量使用连续数组存储。

建议数据流：

```
人工编辑JSONL / CSV
  ↓
构建脚本校验
  ↓
消除重复与编译规则
  ↓
生成浏览器二进制包
  ↓
随PWA发布
```

人工编辑格式和浏览器运行格式应分离。

7. 本地数据存储

固定翻译数据

核心词库是只读数据，优先直接作为静态资源缓存：

```
Cache Storage
```

不需要在每次启动时把整个词库写入数据库。

用户数据

使用IndexedDB保存：

- 翻译历史；
- 收藏；
- 用户自定义词语；
- 场景偏好；
- 更新版本状态；
- 已下载的增量补丁。

IndexedDB适合在浏览器中持久化较大量的结构化数据，并支持离线查询。

不建议使用localStorage

`localStorage` 只适合保存很少的配置，例如：

```
theme=dark  
showJyutping=true  
defaultScene=daily
```

不要用它保存词库或翻译历史。

8. 是否使用SQLite WASM

第一版：不使用

10,000个翻译单元规模较小，翻译时需要频繁进行：

- 最长短语匹配；
- Trie遍历；
- 规则匹配；
- 候选组合；
- N-gram评分。

这些操作在内存数据结构中比不断执行SQL更自然。

推荐：

静态二进制词库
→ Worker加载
→ 内存Trie和TypedArray查询

第二版可选使用

当出现以下需求时，再引入SQLite WASM：

- 词库扩展到几十万条；
- 用户安装多个地区词库包；
- 支持复杂反向查词；
- 允许大规模自定义词库；
- 需要数据表级增量更新；
- 浏览器内提供词条编辑后台。

SQLite官方提供可在现代浏览器中运行的WebAssembly和JavaScript接口，并可使用OPFS保存数据库。

推荐结构：

翻译热路径：
内存Trie+规则

词条管理和更新：
SQLite WASM+OPFS

而不是所有翻译步骤都直接查询SQLite。

9. OPFS和兼容降级

OPFS是网页所属来源的私有文件系统，适合保存SQLite数据库、大型数据包或用户词库；它针对文件原地读写进行了优化，并可在Web Worker中使用。OPFS要求安全上下文。

建议使用三级降级：

优先：
OPFS

不支持OPFS时：
IndexedDB

只读最低兼容模式：
Service Worker Cache

第一版核心翻译功能不应依赖OPFS，否则会不必要地增加浏览器兼容复杂度。

10. Worker通信协议

主线程和翻译Worker之间只传递简单对象。

初始化

```
worker.postMessage({
  type: "INIT",
  dataVersion: "2026.07.1",
  scene: "general"
});
```

翻译请求

```
worker.postMessage({
  type: "TRANSLATE",
  requestId: "req-1024",
  text: "你为什么还没有给我看这个文件？",
  options: {
    scene: "work",
    variant: "hong-kong",
    includeJyutping: false
  }
});
```

翻译结果

```
{
  type: "RESULT",
  requestId: "req-1024",
  source: "你为什么还没有给我看这个文件？",
  target: "你点解仲未畀我睇呢份文件呀？",
  confidence: 0.94,
  matchedTemplate: "WHY_NOT_YET",
  warnings: []
}
```

低置信度结果

```
{
  type: "RESULT",
  requestId: "req-1025",
  target: ".....",
}
```

```
confidence: 0.48,  
warnings: [  
  "包含未覆盖表达",  
  "建议缩短句子"  
]  
}
```

11. 翻译核心是否使用WebAssembly

MVP : TypeScript

第一版建议全部使用TypeScript：

- 开发和调试快；
- 规则错误容易定位；
- 10,000翻译单元无需WASM也能快速运行；
- 数据结构和Worker已经能避免界面卡顿。

V2 : Rust编译为WASM

当出现以下情况时，再把核心迁移到Rust/WASM：

- 词库超过10万条；
- 复杂规则明显增加；
- 候选搜索速度不足；
- 需要与移动原生应用共享同一套引擎；
- 希望核心逻辑不容易被直接修改；
- 需要更严格的内存布局。

WebAssembly是浏览器中的紧凑二进制执行格式，也可作为Rust、C和C++等语言的浏览器编译目标。

推荐保持统一接口：

```
interface TranslatorEngine {  
  init(data: ArrayBuffer[]): Promise<void>;  
  translate(  
    text: string,  
    options: TranslateOptions  
  ): TranslateResult;  
}
```

这样TypeScript版本和WASM版本可以无缝替换。

12. 离线缓存策略

安装阶段

Service Worker首次安装时缓存最小必要资源：

```
HTML
CSS
主程序
Worker
核心5,000条数据
```

安装完成后，应用即可进入基础离线模式。

扩展数据包

其余场景词库可以作为独立包：

```
pack-basic.bin
pack-shopping.bin
pack-transport.bin
pack-work.bin
pack-medical.bin
```

可以选择：

- 首次安装全部缓存；
- 用户按需下载；
- 在有网络时自动下载；
- 企业部署时直接全部预装。

缓存优先策略

应用代码：

```
Cache First
```

版本信息：

```
Network First, 失败时使用缓存
```

翻译数据：

```
Cache First+版本哈希校验
```

13. 词库更新机制

服务器只承担“发布更新包”的作用，翻译过程不访问服务器。

版本清单

```
{
  "engineVersion": "1.3.0",
  "dataVersion": "2026.07.15",
  "packages": [
    {
      "name": "core",
      "version": "37",
      "url": "/data/core-v37.bin",
      "sha256": "...",
      "size": 4821032
    }
  ]
}
```

更新流程

1. 应用检测version.json
2. 比较本地版本
3. 下载新数据包
4. 校验文件哈希
5. 完整加载并执行自检
6. 原子切换到新版本
7. 删除旧缓存

不要边下载边覆盖现有词库。只有新包全部校验成功后才切换。

Service Worker更新时，新版本通常先进入等待状态，因此应用应显示“有新版本可用”，由用户刷新或重新启动后完成切换。

14. 目录结构

```
/
├─ index.html
├─ manifest.webmanifest
├─ service-worker.js
├─ assets/
│   └─ app.js
│   └─ app.css
```



```
|   └─ icons/
├─ worker/
|   └─ translator-worker.js
|       └─ engine.wasm
├─ data/
|   └─ version.json
|   └─ strings.bin
|   └─ phrase.bin
|   └─ lexicon.bin
|   └─ templates.bin
|   └─ rules.bin
|       └─ ngram.bin
└─ licenses/
    └─ third-party-notices.txt
        └─ data-sources.json
```

`engine.wasm` 在TypeScript MVP中可以不存在。

15. 推荐代码模块

```
src/
├─ ui/
|   └─ editor.ts
|   └─ result-view.ts
|       └─ settings.ts
├─ worker/
|   └─ worker-entry.ts
|       └─ protocol.ts
├─ engine/
|   └─ normalizer.ts
|   └─ tokenizer.ts
|   └─ phrase-matcher.ts
|   └─ template-matcher.ts
|   └─ rewrite-engine.ts
|   └─ candidate-generator.ts
|   └─ scorer.ts
|       └─ postprocessor.ts
├─ data/
|   └─ loader.ts
|   └─ binary-reader.ts
|       └─ schema.ts
└─ storage/
    └─ indexed-db.ts
        └─ updates.ts
```

每个模块必须可以独立单元测试。

16. 工程性能目标

对于约10,000个翻译单元，建议将目标设为：

指标	目标
首次完整下载	5—20MB
缓存后启动	1秒内
Worker初始化	300毫秒内
普通短句翻译	50毫秒内
最长输入	50个汉字
候选数量	不超过8个
运行内存	50MB以内
离线可用率	100%核心功能
翻译网络请求	0

这些是工程验收目标，需要在低端Android手机、iPhone、Windows和Mac浏览器上实测。

17. 浏览器兼容策略

核心功能只依赖：

- Service Worker；
- Cache Storage；
- Web Worker；
- IndexedDB；
- ArrayBuffer和TypedArray；
- 可选WebAssembly。

不要让核心翻译依赖：

- SharedWorker；
- 后台同步；
- 浏览器扩展；
- 实验性文件API；
- WebGPU；
- 云端接口。

降级逻辑：

支持Service Worker：
完整PWA离线模式

不支持Service Worker但支持Worker：
当前页面可翻译，但不能保证关闭后离线重开

不支持Worker：
在主线程运行简化翻译引擎

不支持WASM：
使用TypeScript引擎

18. 安全和隐私

网页构建时应做到：

- 不引用CDN脚本；
- 不引用在线字体；
- 不加载远程分析SDK；
- 不调用在线翻译API；
- 所有依赖在构建时打包；
- 设置严格Content Security Policy；
- 用户输入默认不上传；
- 翻译历史只保存在本地；
- 提供“一键清除本地数据”；
- 发布包附带数据来源和许可证清单。

最终页面在断网状态下，开发者工具的Network面板中不应出现任何翻译相关请求。

19. 最终推荐路线

第一阶段

采用：

PWA
+ TypeScript
+ Dedicated Web Worker
+ Trie短语匹配
+ 模板和规则引擎
+ 二进制只读词库
+ IndexedDB用户数据

暂时不采用：

SQLite WASM
神经翻译模型

大型NLP框架
Rust/WASM核心

第二阶段

当数据和规则规模增长后，再升级为：

PWA
+ Rust/WASM翻译核心
+ SQLite WASM词库管理
+ OPFS持久化
+ 量化N-gram排序模型

20. 简化后的技术结论

第一版最合适的浏览器实现是：

Service Worker负责离线
Web Worker负责计算
二进制数据包负责词库
Trie负责短语检索
模板和有限状态规则负责语法转换
轻量评分器负责选择最终粤语结果
IndexedDB负责用户数据

对于一万个翻译单元，这一方案比在浏览器里部署神经翻译模型更小、更快，也更容易人工控制和持续修正规则。